B.Comp. Dissertation

DGLight: DQN-Guided GRPO Fine-Tuning of Large Language Models for Traffic Signal Control

Yu Chenbo

Department of Computer Science
School of Computing
National University of Singapore

2025/2026

B.Comp. Dissertation

DGLight: DQN-Guided GRPO Fine-Tuning of Large Language Models for Traffic Signal Control

Yu Chenbo

Department of Computer Science

School of Computing

National University of Singapore

2025/2026

Project No: H118280

Advisor: Prof. Roger Zimmermann

Main Evaluator: Assoc. Prof. Aaron Tan Tuck Choy

Deliverables:

Report: 1 Volume

Program: 1 Codebase

Abstract

Table of Contents

1 Introduction

Rapid urbanization and accelerating motorization have placed unprecedented strain on urban transportation systems worldwide. As cities densify, limited road infrastructure must accommodate ever-growing travel demand, leading to persistent congestion, unreliable travel times, and increased environmental externalities. Inefficient traffic flow not only wastes fuel and economic productivity but also amplifies air pollution and degrades quality of life, particularly in densely populated urban corridors.

Traffic signal control (TSC) aims to coordinate phase sequences and durations at signalized intersections to maximize throughput and reduce congestion. Classical fixed-time methods use the Webster formula[17] to preset signal timings based on historical traffic data collected from the traffic network at different times. In contrast, the max-pressure control framework[6] computes a pressure value for each signal phase from the difference between queue lengths on opposing approaches and selects the phase with the highest pressure; this strategy can be shown to maximize network throughput.

Reinforcement-learning (RL) methods model TSC as a Markov decision process in which an agent observes traffic states (for example, lane queue lengths and delays), chooses an action (phase switch) and receives a reward linked to throughput or delay reduction. Deep RL further parameterizes the control policy using neural networks, enabling the agent to approximate complex, high-dimensional decision functions. Such methods have demonstrated strong empirical performance across a wide range of simulated traffic scenarios. However, they also introduce notable challenges. In particular, learned policies often exhibit limited generalization, as models trained in one traffic environment exhibit degraded performance when deployed in different cities or under unseen conditions. Furthermore, the reliance on opaque neural representations reduces interpretability, making it difficult to explain or verify decision-making processes. This lack of transparency poses a barrier to real-world adoption, where reliability and accountability are critical.

Large Language Models (LLMs) represent a new paradigm in artificial intelligence, combining strong reasoning ability with the capacity to generate interpretable, text-based outputs. These properties make LLMs promising candidates for traffic signal control, where decision transparency and adaptability are highly desirable. By framing control as a reasoning process, LLM-based agents can potentially provide both actionable policies and human-understandable explanations, bridging the gap between performance and interpretability. However, this potential is not yet fully realized in practice. Achieving real-time responsiveness typically requires deploying smaller-scale models or mixture-of-experts (MoE) architectures with compact expert networks. Without sufficient domain adaptation, such models often fail to capture complex traffic dynamics, leading to suboptimal decisions.

to answer all the demands, I ....

2 Related Work

Traffic Signal Control. Early TSC methods such as *FixedTime* [17] rely on predefined cycle lengths and manually designed phase splits for each intersection. Although these

approaches are simple and stable in practice, they adapt poorly to time-varying traffic demand. *MaxPressure* [6] improves adaptability by selecting phases according to the queue imbalance between incoming and outgoing directions, making it more responsive than fixed plans. However, both *FixedTime* and *MaxPressure* remain constrained by handcrafted rules or preset logic, which limits their flexibility under complex and highly dynamic traffic scenarios.

Recent progress in reinforcement learning has led to a series of RL-based TSC methods. *CoLight* [16] models network-level cooperation among intersections through graph attention, enabling each signal to coordinate with surrounding intersections instead of acting independently. *CoSLight* [10] further extends this idea by jointly optimizing signal control and collaborator selection, so that each intersection can dynamically decide which neighboring intersections are most useful for coordination under current traffic conditions. *MPLight* [1] is a decentralized and pressure-aware RL approach designed for large-scale road networks, using parameter sharing to improve scalability and training efficiency. Although these methods achieve strong performance in per city simulation, RL-based controllers face challenges in generalization and interpretability, hence deployment robustness in real-world environments.

To address these limitations, recent studies have begun exploring the integration of large language models (LLMs) into TSC. *LLM-Assisted Light* [15] places an LLM at the center of control and augments it with perception and decision-support tools so that it can combine live traffic observations with established TSC methods and remain robust under emergency, roadblock, and sensor-outage scenarios. *iLLM-TSC* [8] instead adopts a hybrid design in which an RL controller first proposes an action and an LLM then verifies or adjusts that action when observations are degraded or rare events such as packet loss and emergency vehicles fall outside the original RL formulation. *LATS* [18] further explores a teacher-student setting in which an LLM teacher provides rich semantic features during multi-agent RL training, while a smaller student controller learns to emulate those features and is deployed without the LLM at inference time. *LLMLight* [4] moves closer to end-to-end LLM control by adapting its LightGPT backbone through imitation fine-tuning and critic-guided policy refinement so that the model can make direct signal decisions with improved interpretability and cross-scenario performance. *Traffic-R1* [19] extends this trend with a 3B RL-finetuned foundation model that combines expert-aligned offline training, online self-exploration, and asynchronous multi-agent communication to achieve strong zero-shot and out-of-distribution TSC performance. Compared with end-to-end LLM finetuning, LLM+RL hybrids may remain inferior because they still inherit the ceiling and failure modes of the underlying RL controller. Existing end-to-end finetuning methods also have notable limitations: In *LLMLight*, the critic-guided policy refinement objective is weaker than modern policy-gradient methods because the reward/advantage magnitude does not directly participate in loss computation. *Traffic-R1*, meanwhile, relies on substantial human effort to label the offline finetuning data, and its fixed-horizon cumulative queue-based reward assigns the same coarse mean reward to all steps within the horizon.

RL finetuning of LLMs. RL-based preference alignment [7] adapts a pretrained LLM using scalar or pairwise feedback so that the policy is optimized toward preferred responses rather than merely imitating target outputs. Among modern policy optimization methods, TRPO [12] constrains updates within a trust region to stabilize improvement, PPO [11] replaces this with a clipped surrogate objective that is simpler and more prac-

tical at scale, and GRPO [13] further removes the separate value model by computing group-relative advantages over sampled responses, which makes it especially attractive for LLM training. Closely related to these RL-style methods, DPO [9] performs direct preference optimization on chosen–rejected response pairs and avoids explicit reward-model fitting and online rollout collection. This line of work is particularly important because preference-aligned RL can train reasoning ability rather than only answer imitation: through exploration, comparison, and preference-based updates, the model learns to assign higher probability to response trajectories that exhibit stronger multi-step reasoning while remaining close to a reference policy through KL regularization, as emphasized in RLHF-style alignment and recent reasoning-oriented RL studies [7, 13, 3]. Recent systems such as *DeepSeek-R1* [3] show that large-scale self-exploration can substantially improve reasoning behavior, making RL preference alignment increasingly relevant for domain-specific agents that must optimize long-horizon objectives.

3 Preliminaries

Definition 1. ***Road Network.*** A road network is represented as a directed graph consisting of intersections $\mathcal{V}$ and lanes $\mathcal{L}$. The lane set is grouped into three categories: go-through lanes ($\mathcal{L}_{\text{go}}$), left-turn lanes ($\mathcal{L}_{\text{left}}$), and right-turn lanes ($\mathcal{L}_{\text{right}}$). Each lane connects adjacent intersections and is partitioned into segments $S = \{s_1, \ldots, s_n\}$ according to its distance from the intersection.

Definition 2. ***Signal Phase.*** At each signal-switching step, the controller of an intersection chooses a signal phase from a predefined action set $A = \{a_1, \ldots, a_m\}$. A phase is written as $a = \text{set}(\mathcal{L}_{\text{allow}})$, where $\mathcal{L}_{\text{allow}}$ denotes the collection of lanes that are allowed to move simultaneously without conflict, meaning these lanes receive green signals while conflicting lanes remain red.

Definition 3. ***Traffic Signal Control System.*** The traffic signal control system consists of multiple agents $\Pi = \{\pi_1, \ldots, \pi_n\}$, with each agent assigned to one of the n intersections in the road network. For intersection i, agent π_i makes its signal control decision by observing the number of vehicles in each segment of the lanes approaching that intersection, thereby forming a segment-level view of the local traffic state.

Problem Statement. ***LLM Reasoning for Traffic Signal Control.*** For each intersection i in the road network, an LLM serves as agent π_i. At decision step t, the agent takes the observation $O_{i,t}$, the task description d_{task}, and the action space A as input, and outputs both reasoning content and a signal phase decision. The TSC problem is therefore formulated as selecting a phase $a_{i,t} \in A$ while generating reasoning output $R_{i,t}$ under the given observation and task specification. The system dynamics are defined as:

$$(R_{i,t}, a_{i,t}) = \pi_{\theta_i}(O_{i,t}, d_{\text{task}}, A) \tag{1}$$

where $O_{i,t}$ is the observation, d_{task} is the task description, A is the action space, $R_{i,t}$ is the reasoning output, and $a_{i,t}$ is the selected phase.

Two-stage pipeline overview.
Stage 1: DQN critic pretraining. Train a CoLight-based DQN critic that estimates per-phase action values from graph-structured traffic observations.
Stage 2: DGLight fine-tuning. (1) Convert each intersection state into a textual prompt. (2) Sample k candidate responses from the LLM. (3) Parse each response into a signal phase. (4) Query the frozen critic for per-action Q-values. (5) Score the sampled candidates and execute the best sampled action in CityFlow. (6) Store prompt–score records for all intersections and time steps. (7) Run GRPO on the collected records to update the LLM policy.

Figure 1: Overview of the two-stage *DGLight* pipeline. An alternative joint-scored comparison variant is described separately in Appendix A.0.3.

4 Methodology

This section introduces *DGLight*, a two-stage reinforcement-learning pipeline for adapting a pretrained large language model to traffic signal control. In the first stage, a CoLight-based DQN critic is trained to estimate action values for signal-phase decisions when provided with structured environment states. In the second stage, the frozen critic scores candidate LLM responses during rollout and the scores are used as the reward signal for GRPO-based policy optimization.

4.1 Pretraining the CoLight-Based DQN Critic

The critic used in *DGLight* is based on *CoLight* [16], a graph-attentive deep Q-learning model for network-level traffic signal control. *CoLight* is employed as a pretrained value estimator so that the subsequent GRPO stage can query a traffic-aware action-value function rather than rely exclusively on the raw environment reward. Given the current environment state, the critic predicts action values for each intersection and thereby provides a dense estimate of the quality of candidate signal phases under the prevailing traffic situation.

The *CoLight* critic is composed of three main components as illustrated in Figure 2. The first component is an observation embedding module that transforms the local traffic state of each intersection into a latent feature representation. The second component consists of one or more multi-head graph-attention layers, which propagate information across neighboring intersections and produce attention-weighted network representations. The final component is a Q-value prediction layer that maps the aggregated features to one scalar value for each admissible signal phase. This architecture combines local state encoding, inter-intersection coordination, and value-based action estimation within a single model.

Training follows the standard value-based reinforcement learning procedure. The model interacts with the traffic environment to collect state transitions, actions, rewards, and subsequent states $((s, a, r, s'))$, where the stepwise reward is defined to be inversely proportional to queue lengths at each intersection. These transitions are stored in replay

Critic architecture placeholder.
Replace this box with a figure showing: local traffic observations → observation embedding layers → graph attention over neighboring intersections → per-phase Q-value prediction head.

Figure 2: Mockup placeholder for the *CoLight*-based DQN critic architecture used in *DGLight*.

memory for off-policy updates. The optimization objective minimizes the Bellman error

$$\mathcal{L}_{\text{Bellman}}(\theta) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\left[\left(r + \gamma \max_{a'} Q_{\theta^-}(s', a') - Q_\theta(s, a)\right)^2\right],$$

where $\mathcal{D}$ denotes the replay buffer, θ denotes the online network parameters, and θ^- denotes the target network parameters. Experience replay improves sample efficiency, and the target network improves optimization stability.

Disclaimer: The critic model, its training objective, and its overall learning logic are not original contributions of this dissertation; they are inherited from prior *CoLight* research. My contribution to this component is limited to re-implementing the model in PyTorch for this codebase, whereas the original *CoLight* implementation was based on TensorFlow.

4.2 Prompted LLM Policy for Intersection Control

At each signal-switching step t, each intersection i is converted into a textual prompt $x_{i,t}$ that summarizes the current traffic condition together with the task instruction and the available signal phases. The prompt is constructed from the environment state used by the controller, including the current phase and traffic movement pressure/queue information, and is formatted so that the model produces both human-readable reasoning and a final phase choice. This design preserves interpretability while remaining fully compatible with autoregressive language modeling.

Given prompt $x_{i,t}$, the policy π_θ samples a group of k candidate responses

$$\{y_{i,t}^{(1)}, y_{i,t}^{(2)}, \ldots, y_{i,t}^{(k)}\}.$$

Each response is then mapped to a signal phase through a deterministic parser:

$$a_{i,t}^{(j)} = \text{Parse}(y_{i,t}^{(j)}), \qquad j = 1, \ldots, k.$$

If the response fails to specify a valid action in the predefined action space, it is treated as an invalid output and assigned a fallback reward during training. This allows the method to preserve free-form reasoning generation while still enforcing a discrete control interface at execution time.

4.3 DQN-Guided Action Scoring

The central idea of *DGLight* is to replace coarse trajectory-level rewards with direct action-level evaluation from the pretrained DQN critic. For the current traffic state at

intersection i, the frozen critic produces a vector of estimated returns over all admissible signal phases:

$$\mathbf{q}_{i,t} = \big[Q_\phi(s_{i,t}, a)\big]_{a \in A},$$

where $s_{i,t}$ is the structured traffic state and Q_ϕ is the pretrained DQN critic. The critic therefore acts as a traffic-aware evaluator that maps the current state to a dense preference ordering over actions.

Each sampled LLM response is scored by the critic through its parsed action:

$$r_{i,t}^{(j)} = \begin{cases} Q_\phi\Big(s_{i,t}, a_{i,t}^{(j)}\Big), & \text{if } a_{i,t}^{(j)} \in A, \\ r_{\text{invalid}}, & \text{otherwise}, \end{cases}$$

where r_{invalid} is a fixed fallback reward for malformed or out-of-range decisions. Among the sampled candidates, the action actually executed in the environment is the highest-scoring sampled action,

$$j_{i,t}^{*} = \arg\max_{j \in \{1,\dots,k\}} r_{i,t}^{(j)}, \qquad \hat{a}_{i,t} = a_{i,t}^{(j_{i,t}^{*})}.$$

Thus, the environment rollout is still driven by sampled LLM outputs, but execution is guided by the critic's action-value estimate.

This design yields two methodological advantages. First, the reward signal is dense: every sampled response receives an immediate action-specific score rather than waiting for a sparse terminal outcome. Second, the reward is directly aligned with the control decision itself, since the model is encouraged to generate reasoning trajectories whose final answer corresponds to a high-value traffic action. Compared with fixed-horizon cumulative congestion rewards, this provides finer credit assignment at each decision step. Compared with ranking-style refinement objectives, the magnitude of the action preference directly influences the GRPO update.

4.4 GRPO-Based Policy Optimization

After rollout collection, each intersection-step pair contributes a training record consisting of the prompt and the full critic score vector,

$$\mathcal{D}_{i,t} = \big(x_{i,t}, \mathbf{q}_{i,t}\big).$$

Importantly, the stored supervision is not just the score of the executed action. Instead, the entire per-action vector is retained so that any newly sampled response for the same prompt can be rewarded according to the action it proposes. During GRPO training, a completion y sampled for prompt $x_{i,t}$ is assigned reward

$$R(x_{i,t}, y; \mathbf{q}_{i,t}) = \begin{cases} \mathbf{q}_{i,t}[a(y)], & \text{if } a(y) \in A, \\ r_{\text{invalid}}, & \text{otherwise}, \end{cases}$$

where $a(y)$ denotes the parsed action of completion y. The resulting reward values are then normalized within each sampled group, producing relative advantages that drive the GRPO update.

This optimizer is particularly well suited to the present setting. Unlike PPO-style methods that typically require a separate learned value model, GRPO estimates relative advantages directly from groups of sampled responses for the same prompt. This makes it a natural fit for LLM fine-tuning, where multiple completions can be generated efficiently for a single traffic state. More importantly, it addresses the gap identified in Section 2: instead of relying on a weaker ranking-only refinement stage, the policy is updated through a modern RL objective in which critic-derived reward directly shapes the optimization signal.

4.5 End-to-End DGLight Training Procedure

The complete training loop can be summarized as follows:

1. Pretrain the CoLight-based DQN critic.
2. Freeze the critic checkpoint for downstream use in *DGLight*.
3. Reset the CityFlow environment and build one textual prompt for each active intersection.
4. Sample k candidate responses from the LLM for every prompt.
5. Parse each response into a discrete signal phase.
6. Query the frozen critic to obtain per-action Q-values for the current traffic state.
7. Score the sampled responses according to their parsed actions and execute the best sampled action at each intersection.
8. Store the prompt together with the full per-action score vector as a GRPO training record.
9. Repeat the above process for the full rollout episode, then fine-tune the policy on the collected records with GRPO.

In practice, the two stages are decoupled. The critic is first trained offline as a CoLight-based DQN value estimator, and the resulting checkpoint is then reused as a fixed evaluator during LLM fine-tuning. Exact hyperparameter settings, model sizes, rollout lengths, and dataset configurations are deferred to the experimental section.

4.6 Comparison Variant

For completeness, I also implemented a *Joint-Scored GRPO* variant as a comparison method. Instead of using direct critic Q-values to score sampled actions, that variant estimates action quality indirectly from short-horizon joint rollout returns over snapshot-forked simulations, and then projects those returns back to per-intersection action scores for GRPO training. Because this variant is included only to test whether indirect joint scoring can outperform direct critic guidance, I do not treat it as a co-equal methodological contribution in the main text. Its detailed design is provided separately in Appendix A.0.3.

5 References

References

[1] Chacha Chen et al. "Toward A Thousand Lights: Decentralized Deep Reinforcement Learning for Large-Scale Traffic Signal Control." In: *Proceedings of the AAAI Conference on Artificial Intelligence* 34.04 (Apr. 2020), pp. 3414–3421. DOI: 10.1609/aaai.v34i04.5744. URL: https://ojs.aaai.org/index.php/AAAI/article/view/5744.

[2] Lang Feng et al. *Group-in-Group Policy Optimization for LLM Agent Training*. 2025. arXiv: 2505.10978 [cs.LG]. URL: https://arxiv.org/abs/2505.10978.

[3] Daya Guo et al. *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*. 2025. arXiv: 2501.12948 [cs.CL]. URL: https://arxiv.org/abs/2501.12948.

[4] Siqi Lai et al. *LLMLight: Large Language Models as Traffic Signal Control Agents*. 2024. arXiv: 2312.16044 [cs.AI]. URL: https://arxiv.org/abs/2312.16044.

[5] Pablo Alvarez Lopez et al. "Microscopic Traffic Simulation using SUMO." In: *The 21st IEEE International Conference on Intelligent Transportation Systems*. IEEE, 2018. URL: https://elib.dlr.de/124092/.

[6] Pedro Mercader, Wasim Uwayid, and Jack Haddad. "Max-pressure traffic controller based on travel times: An experimental analysis." In: *Transportation Research Part C: Emerging Technologies* 110 (2020), pp. 275–290. ISSN: 0968-090X. DOI: https://doi.org/10.1016/j.trc.2019.10.002. URL: https://www.sciencedirect.com/science/article/pii/S0968090X19307442.

[7] Long Ouyang et al. *Training Language Models to Follow Instructions with Human Feedback*. 2022. arXiv: 2203.02155 [cs.CL]. URL: https://arxiv.org/abs/2203.02155.

[8] Aoyu Pang et al. *iLLM-TSC: Integration Reinforcement Learning and Large Language Model for Traffic Signal Control Policy Improvement*. 2024. arXiv: 2407.06025 [cs.AI]. URL: https://arxiv.org/abs/2407.06025.

[9] Rafael Rafailov et al. *Direct Preference Optimization: Your Language Model is Secretly a Reward Model*. 2023. arXiv: 2305.18290 [cs.LG]. URL: https://arxiv.org/abs/2305.18290.

[10] Jingqing Ruan et al. *CoSLight: Co-optimizing Collaborator Selection and Decision-making to Enhance Traffic Signal Control*. 2024. arXiv: 2405.17152 [cs.MA]. URL: https://arxiv.org/abs/2405.17152.

[11] John Schulman et al. *Proximal Policy Optimization Algorithms*. 2017. arXiv: 1707.06347 [cs.LG]. URL: https://arxiv.org/abs/1707.06347.

[12] John Schulman et al. *Trust Region Policy Optimization*. 2015. arXiv: 1502.05477 [cs.LG]. URL: https://arxiv.org/abs/1502.05477.

[13] Zhihong Shao et al. *DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models*. 2024. arXiv: 2402.03300 [cs.CL]. URL: https://arxiv.org/abs/2402.03300.

[14] Zheng Tang et al. *CityFlow: A City-Scale Benchmark for Multi-Target Multi-Camera Vehicle Tracking and Re-Identification*. 2019. arXiv: 1903.09254 [cs.CV]. URL: https://arxiv.org/abs/1903.09254.

[15] Maonan Wang et al. *LLM-Assisted Light: Leveraging Large Language Model Capabilities for Human-Mimetic Traffic Signal Control in Complex Urban Environments*. 2024. arXiv: 2403.08337 [eess.SY]. URL: https://arxiv.org/abs/2403.08337.

[16] Hua Wei et al. "CoLight: Learning Network-level Cooperation for Traffic Signal Control." In: *Proceedings of the 28th ACM International Conference on Information and Knowledge Management*. CIKM '19. Beijing, China: Association for Computing Machinery, 2019, pp. 1913–1922. ISBN: 9781450369763. DOI: 10.1145/3357384.3357902. URL: https://doi.org/10.1145/3357384.3357902.

[17] Biao Yin, Mahjoub Dridi, and Abdellah Moudni. "Traffic network micro-simulation model and control algorithm based on approximate dynamic programming." In: *IET Intelligent Transport Systems* 10 (Dec. 2015). DOI: 10.1049/iet-its.2015.0108.

[18] Yifeng Zhang et al. *LATS: Large Language Model Assisted Teacher-Student Framework for Multi-Agent Reinforcement Learning in Traffic Signal Control*. 2026. arXiv: 2603.24361. URL: https://arxiv.org/abs/2603.24361.

[19] Xingchen Zou et al. *Traffic-R1: Reinforced LLMs Bring Human-Like Reasoning to Traffic Signal Control Systems*. 2025. arXiv: 2508.02344 [cs.AI]. URL: https://arxiv.org/abs/2508.02344.

A Appendix

A.0.1 Reinforcement Learning Overview

Reinforcement learning (RL) models sequential decision making as a Markov decision process (MDP) $(\mathcal{S}, \mathcal{A}, p, r, \gamma)$. At each time step t, the agent observes a state $s_t \in \mathcal{S}$, selects an action $a_t \in \mathcal{A}$ according to a policy $\pi_\theta(a_t \mid s_t)$, receives a scalar reward $r_t = r(s_t, a_t)$, and the environment transitions to $s_{t+1} \sim p(\cdot \mid s_t, a_t)$. The objective is to find policy parameters θ that maximize the expected discounted return

$$J(\theta) = \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{\infty} \gamma^t r_t\right],$$

where $\gamma \in (0, 1]$ is the discount factor. For a fixed policy π, the state-value function and action-value function are

$$V^\pi(s) = \mathbb{E}_\pi\big[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s\big], \qquad Q^\pi(s, a) = \mathbb{E}_\pi\big[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s, a_0 = a\big],$$

and they satisfy the Bellman expectation equation

$$V^\pi(s) = \sum_a \pi(a \mid s) \sum_{s'} p(s' \mid s, a)\big[r(s, a) + \gamma V^\pi(s')\big].$$

The advantage function is defined as

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s),$$

and measures how much better action a is compared to the average action at state s.

Policy gradient principle. For differentiable policies $\pi_\theta(a \mid s)$, the policy gradient theorem gives

$$\nabla_\theta J(\theta) = \mathbb{E}_{s \sim d^\pi, a \sim \pi_\theta}\big[\nabla_\theta \log \pi_\theta(a \mid s)\, Q^\pi(s, a)\big],$$

and in practice we replace $Q^\pi(s, a)$ by an advantage estimate $\hat{A}_t$, leading to the classic REINFORCE-style update

$$\theta \leftarrow \theta + \alpha\, \hat{A}_t\, \nabla_\theta \log \pi_\theta(a_t \mid s_t).$$

Most modern RL methods (PPO, GRPO, and their variants) can be viewed as safer or more sample-efficient ways of performing this update.

Proximal Policy Optimization (PPO). PPO [11] stabilizes policy gradients by preventing excessively large policy updates. Let

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}$$

be the probability ratio between the new and the behavior policy. Let $\hat{A}_t$ be an advantage estimate (e.g., GAE). PPO maximizes the clipped surrogate objective

$$L^{\mathrm{PPO}}(\theta) = \mathbb{E}_t\Big[\min\big(r_t(\theta)\hat{A}_t,\ \mathrm{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\big)\Big],$$

where ϵ is a small hyperparameter (e.g., 0.1 or 0.2). The $\min(\cdot)$ and the clipping together act as a trust region in probability space, so that even if $\hat{A}_t$ is large, the policy will not move too far away from $\pi_{\theta_{\mathrm{old}}}$ in a single update.

Group Relative Policy Optimization (GRPO). Recent LLM-oriented RL work observes that when many responses can be sampled for the same prompt, we can estimate advantages without a separate value network. GRPO forms a group $\mathcal{G} = \{r_1, \ldots, r_N\}$ of rollouts for the same input, scores them with a reward model $R_\phi(\cdot)$, and then normalizes within the group:

$$A_i = \frac{R_\phi(r_i) - \mathrm{mean}(\mathcal{G})}{\mathrm{std}(\mathcal{G})}.$$

This gives a critic-free advantage estimate. GRPO then uses essentially the same clipped PPO-style loss

$$L^{\mathrm{GRPO}}(\theta) = \frac{1}{N}\sum_{i=1}^{N} \min\left(\frac{\pi_\theta(r_i \mid x)}{\pi_{\theta_{\mathrm{old}}}(r_i \mid x)}A_i, \mathrm{clip}\left(\frac{\pi_\theta(r_i \mid x)}{\pi_{\theta_{\mathrm{old}}}(r_i \mid x)}, 1-\epsilon, 1+\epsilon\right)A_i\right) - \lambda\, D_{\mathrm{KL}}\big(\pi_\theta \,\|\, \pi_{\mathrm{ref}}\big),$$

where the KL term keeps the updated policy close to a reference model (often the SFT model). Compared to PPO, the key differences are: (i) advantages come from groupwise normalization, not a learned value function; (ii) the method is well suited to large-batch LLM training where many samples per prompt are available.

Group-in-Group Policy Optimization (GiGPO). Very recent work on training LLM agents in multi-step environments argues that a single group-level relative advantage (as in GRPO) is not enough, because the reward can be sparse and credit assignment needs to happen both at the trajectory level and at individual steps. GiGPO [2] introduces a two-level grouping:

- at the episode level, sample multiple full trajectories for the same task and compute a macro relative advantage (similar to GRPO);
- at the step level, retroactively group actions that occurred in the same environment state (called anchor states) across different trajectories, and compute a micro relative advantage for those actions.

This yields fine-grained credit signals without introducing a value network. Conceptually, GiGPO keeps the good parts of GRPO (critic-free, low memory, PPO-like clipping) but adds an extra axis of grouping so that actions that happened at the same state can be compared directly. This is particularly attractive for long-horizon urban/traffic tasks or LLM agents, where the effect of a single bad action might only become visible many steps later.

A.0.2 Traffic Signal Control

General overview. Traffic signal control (TSC) aims to coordinate phase sequences and durations at signalized intersections to maximize throughput and reduce congestion. Classical fixed-time methods use the Webster formula[17] to preset signal timings based on historical traffic data collected from the traffic network at different times. In contrast, the max-pressure control framework[6] computes a pressure value for each signal phase from the difference between queue lengths on opposing approaches and selects the phase with the highest pressure; this strategy can be shown to maximize network throughput. Reinforcement-learning (RL) methods model TSC as a Markov decision process in which an agent observes traffic states (for example, lane queue lengths and delays), chooses an action (phase switch) and receives a reward linked to throughput or delay reduction. CoLight[16] learns network-level cooperation by letting intersections communicate through graph attention, so signals can coordinate beyond immediate neighbors. CoSLight[10] extends this idea by learning not only the signal policy but also which intersections should collaborate at each moment, improving multi-intersection coordination under changing congestion patterns. MPLight[1] is a decentralized, pressure-aware RL variant built to run efficiently in large CityFlow[14]-style networks, using parameter sharing to keep city-scale control tractable. Despite achieving competitive performance in simulation, many RL-based controllers suffer from poor cross-city generalization and lack interpretability, motivating recent research into interpretable and generalizable TSC agents.

CityFlow environment. A key enabler for modern RL research on TSC is the availability of scalable simulators. SUMO[5], the traditional open-source traffic simulator, is often too slow for city-scale RL experiments. CityFlow[14] is a multi-agent microscopic traffic simulator designed for large-scale RL studies. It provides flexible definitions of road networks and traffic flows from both synthetic and real-world data and exposes a user-friendly API for interaction with RL agents. CityFlow's multithreaded design makes it more than twenty times faster than SUMO, enabling efficient simulation of networks with hundreds or thousands of intersections. Because of its speed and extensibility, CityFlow has become a standard environment for evaluating RL-based TSC algorithms.

***LLMLight*: Large language models as TSC agents.** The *LLMLight*[4] framework treats traffic signal control as a partially observable Markov game and leverages large language models (LLMs) to serve as decision-making agents. At each signal switch, an agent collects observation features such as the number of queuing vehicles and approaching vehicles on each incoming lane, augments this information with a traffic scenario description, a task description and a commonsense knowledge prompt that encourages the agent to prioritize long queues, and feeds the combined text to an LLM. The LLM performs chain-of-thought reasoning to decide which phase to activate and provides a human-readable rationale. *LLMLight* constructs a specialized backbone model called LightGPT via imitation fine-tuning on GPT-4 trajectories and critic-guided policy refinement. During imitation, the model minimizes the negative log-likelihood

$$\mathcal{L}_{\text{NLL}} = -\sum_{t=1}^{T} \log p_\theta\big(y_t \mid y_{<t}, \pi, \mathcal{P}\big),$$

where π is the prompt consisting of observation and commonsense instructions and $\mathcal{P}$ denotes the reasoning trajectory. A critic model derived from an action-value network evaluates sampled trajectories, and a ranking-based loss steers the model toward trajectories with higher long-term rewards. Experimental results show that *LLMLight* with LightGPT achieves competitive or superior performance and provides better interpretability compared with traditional RL baselines across diverse traffic scenarios. It provides a reproducible workflow that integrates CityFlow with RL-based fine-tuning of LLMs (though implemented in TensorFlow) and datasets ready for use.

***Traffic-R1*: Reinforced LLMs for human-like TSC.** *Traffic-R1*[19] is a 3B foundation model trained to perform traffic signal control with human-like reasoning. It adopts a two-stage reinforcement-learning fine-tuning paradigm: an offline stage aligns the model with question-answer (QA) samples (generated by DeepSeek-R1 and verified by human experts), and an online stage allows self-exploration in simulation to refine the policy via reward feedback. The reward is calculated from the cumulative queuing time after multiple steps, and each step only half of the agents are activated, whose decisions are propagated to neighboring agents as an input. This mimics an asynchronous communication network allows multiple agents to coordinate across intersections. Benchmark results report that *Traffic-R1* offers zero-shot generalization to unseen road networks and out-of-distribution incidents, outperforming both classical heuristics and RL controllers. When deployed in production, *Traffic-R1* manages signals affecting more than 55,000 drivers each day, reducing average queue lengths by over 5% and significantly lowering operator workload. Though without a codebase, *Traffic-R1* provides insights into using SOTA RL algorithms like GRPO to train TSC agents, as well as the innovative idea of mimicking the real world scenario with a asynchronous mult-agent setup.

A.0.3 Joint-Scored GRPO Comparison Variant

This appendix summarizes the *Joint-Scored GRPO* pipeline used as a comparison variant to the proposed *QGuided-GRPO* method. The purpose of this variant is to test whether indirect short-horizon joint return estimation can provide a better training signal than direct critic guidance. Unlike the main method, it does not rely on a pretrained action-value critic to score each sampled response directly.

Snapshot-forked joint candidate evaluation. At each decision step, the policy samples k candidate responses for every intersection and parses them into actions. Instead of scoring each sampled action independently, the method aligns candidates across intersections into k joint action vectors and evaluates each vector by forking the current simulator snapshot. Each fork starts from the same environment state but executes a different sampled joint action, allowing the method to compare candidate action sets under a common traffic condition.

Short-horizon discounted return. For each forked candidate c, the simulator is rolled out for a short horizon H. After the first jointly sampled action is executed, subsequent actions are resampled from the current policy at later steps. The candidate

score is the discounted return

$$G_t^{(c)} = \sum_{\tau=0}^{H-1} \gamma^\tau r_{t+\tau}^{(c)},$$

where γ is the discount factor and $r_{t+\tau}^{(c)}$ is the congestion-based network reward observed at rollout step $t+\tau$ in candidate fork c.

Local, neighborhood, and global reward mixing. The per-step reward is computed from a mixed congestion cost that combines local intersection congestion, neighboring congestion, and global network congestion. Let m_i denote the local congestion metric at intersection i (for example, incoming queue length or pressure), let $\bar{m}_{\mathcal{N}(i)}$ denote the mean metric over the neighbors of i, and let $\bar{m}_{\mathcal{V}}$ denote the mean metric over the whole network. The reward is defined from the negative mixed cost

$$r_t = -\frac{1}{|\mathcal{V}|} \sum_{i \in \mathcal{V}} \Big(\alpha m_i + \beta \bar{m}_{\mathcal{N}(i)} + (1-\alpha-\beta) \bar{m}_{\mathcal{V}} \Big),$$

where α and β control the relative contribution of local and neighborhood congestion. This construction encourages decisions that reduce both immediate local congestion and broader network imbalance.

Projection back to per-intersection action scores. Because GRPO training still expects prompt-level rewards for individual sampled responses, the joint candidate returns must be projected back to action-level scores. For each intersection i and each action $a \in A$, let $\mathcal{C}_{i,a}$ be the set of joint candidates whose parsed action at intersection i equals a. The induced action score is defined as the mean return of those candidates:

$$\tilde{Q}_{i,t}(a) = \frac{1}{|\mathcal{C}_{i,a}|} \sum_{c \in \mathcal{C}_{i,a}} G_t^{(c)},$$

with a fallback reward used when no sampled candidate proposes action a. The resulting vector $\tilde{\mathbf{q}}_{i,t}$ plays the same role in GRPO training as the critic score vector $\mathbf{q}_{i,t}$ in the main method, but it is obtained indirectly through joint rollout evaluation.

Collect-then-train split. In implementation, the variant is organized as a collect-then-train pipeline. A rollout stage first saves prompt records, selected actions, and projected action-score vectors obtained from snapshot-forked candidate evaluation. The GRPO stage then reloads these records and fine-tunes the LLM policy using the same prompt-conditioned reward lookup mechanism as in the main method. This design isolates the additional cost of joint candidate evaluation from the policy optimization stage and makes it suitable for controlled comparison against *QGuided-GRPO*.